



Vending Machine

SOFT2412: Agile Software Development Practices
Sprint 3 Sprint Report

Contents

1	Executive Summary	1
2	Scrum Development	1
2.1	The Team	1
2.2	Project Roadmap	1
2.3	Sprint Goals & Overview	2
2.4	Additional Client Requirements	3
2.5	Product and Sprint Task Boards	3
2.5.1	Priority Hierarchy	4
2.5.2	Assigning Git Commits to User Stories	4
3	Scrum and Agile Development Practices	5
3.1	Agile	5
3.2	Scrum	6
3.2.1	Sprint 3 Review - 03/11/2022	7
3.2.2	Product Backlog	10
3.2.3	Sprint Backlog	10
3.3	Artefacts	10
4	Application Development	12
4.1	Features and Functionality	12
4.2	Final Test Case Production	13
4.2.1	Test Reports	13
4.2.2	Exclusions from Test Reports	13
4.3	Application Design	14
4.3.1	Front-end user interface	14
5	Overall Reflection	17
A	UML diagram	18
B	Meeting Minutes	21

1 Executive Summary

In Sprint 3 the main objectives were to complete user stories left in the backlog and the additional requirements the client had set. This included prioritising generating all the reports and other key requirements from the specification that had not been implemented yet. The client had also reassessed part of the specification and requested implementing security features. Development in this sprint revolved around developing the most complete, deployable application possible.

2 Scrum Development

2.1 The Team

Table 1: Distribution of roles in the scrum team

Member	Role	SID
Sulav Malla	Product Owner	500495980
Udit Samant	Scrum Master	500700976
Antriksh Dhand	Developer	510415022
Nemo Gage	Developer	500496851
Ankit Kapoor	Developer	510470180

2.2 Project Roadmap

The project roadmap ended with this sprint. Therefore, it was referred to throughout the sprint to ensure all epics had been completed. The roadmap is visible in Figure 1.

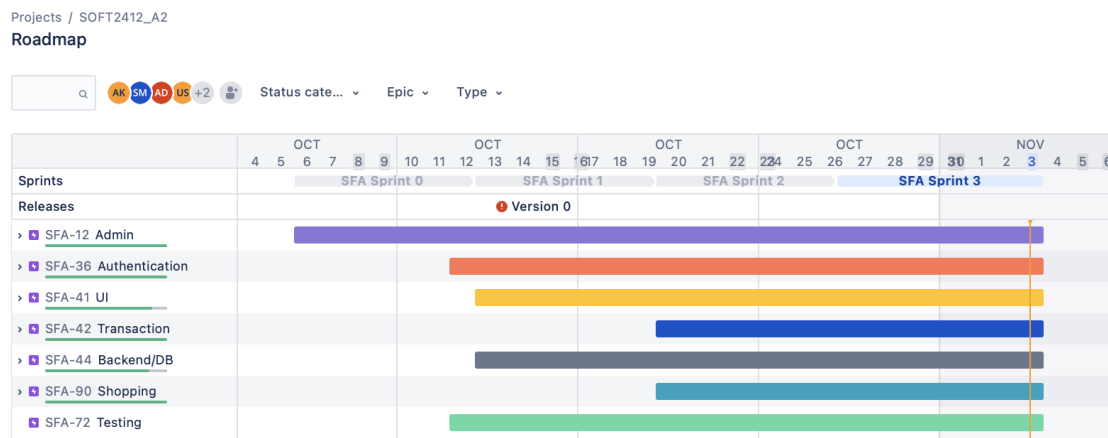


Figure 1: The roadmap for the project, with all epics ending at the end of Sprint 3.

2.3 Sprint Goals & Overview

Sprint 3 Goal

To create a fully functional and deployable application, and to complete *management* functionalities so that owner, seller, and cashier are able to maintain the application.

The exact user stories which were planned for this sprint are listed below. These can also be seen in Figure 5.

User stories for Sprint 3

- As a seller, I want to be able to modify the quantity of items in the vending machine so I manage inventory
- As a seller, I want to be able to produce a summary report of all the items and the quantity sold so far of each item.
- As a seller, I want to be able to produce a summary report of all the items currently in the vending machine, so I may modify items if necessary.
- As a cashier, I want to be able to produce a summary report of all successful transactions that have occurred so far.
- As a cashier, I want to be able to view and update the amount of each denomination in the vending machine so that customers can pay with cash
- As a cashier, I want to be able to produce a summary report of all available notes and coins in the vending machine, so I may update accordingly.
- As an owner, I want to be able to view all of my transactions in a tabular (CSV) format, and for all past transactions to persist until deleted.
- As an owner, I want to be able to add another user of any role so that I can delegate access levels
- As an owner, I want to remove a user from the vending machine application, so that I can manage when a user is inappropriately using the vending machine.
- As an owner, I want my client's details to be encrypted using a salted hash to maximise their security
- As an owner, I want to make sure that the inputted card numbers are valid and from the recognised list of values.
- As an owner, I want to be able to produce a report summarising all the cancelled transaction that have occurred.

- As a user, I want to be able to check-out so I can choose whether to pay with cash or card
- As a user paying with cash, I want to be able to input a specific amount of currency so that I can pay for my order
- As a user, I want to go back to my default page after I start a transaction, so I may log out or purchase more if I want.
- As a user, I want to complete transaction so I can see whether the transaction was successful.
- As a user purchasing with card, I want my card numbers to be stored so that it autofills on my next transaction.
- As a new customer (guest or registered) I want the previous customer's cart to be cleared and logged out should they have logged in.

2.4 Additional Client Requirements

In particular, the following user stories were additional requirements that were presented as ideas to the client at the end of the last sprint.

User Ideas Requested by Client

- As a registered customer or a customer that would like to register I would like my password stored securely so that it is kept safe.
- As an owner, I want to be able to delete a user from the application, so that I can remove redundant accounts.

Both of these requirements were successfully implemented and tested by the end of the sprint. A SHA512 hash was used to encrypt and securely store the passwords of all users.

2.5 Product and Sprint Task Boards

Due to the sheer quantity of features added to the application during this sprint, the project board and backlog were crucial in ensuring the team's success. Therefore, the project board was refined at the start of Sprint 3 to utilise all of the features Jira has to offer to streamline the development process.

2.5.1 Priority Hierarchy

Jira's task priority feature was used to implement a hierarchy to inform team members on which tasks were most urgent. This ensured the most crucial user stories were completed before any additional features were implemented.

The hierarchy included the following priority levels, from highest to lowest priority respectively:

- Highest
- High
- Medium
- Low
- Lowest

Features that were started in the previous sprint but had not been fully completed were assigned the highest priority to ensure that all of the tasks from Sprint 2 were complete. This wasn't an issue, as the team realised at the start of the sprint that there were no lasting tasks that had to be completed, only new ones to be done in this sprint.

Requirements outlined by the client at the start of the project (in the assessment criteria) were given highest priority, with ideas and client requests being given high priority. As per the Agile Manifesto's belief that documentation is not as crucial in agile development as it is in other approaches, tasks to do with drawing and sketching pages were assigned low priority to ensure actual development remained at the top of the hierarchy.

2.5.2 Assigning Git Commits to User Stories

By religiously assigning commits to user stories that were in the 'check requirements' or 'done' tabs, the team was able to see exactly when each story had been completed. This streamlined the process of moving stories out of the 'check requirements' tab and ensured that in case a particular function was causing the program to crash or fail, the team could easily locate the commit that the repository needed to be reverted to.

3 Scrum and Agile Development Practices

3.1 Agile

This sprint agile development practices were used to refine the project. Over the last three sprints agile development practices have improved significantly. There have been major improvements to the efficiency of deployment. Firstly, the team has increased utilisation of git and GitHub while programming. In the first sprint the team was only pushing once a day, and now this increased to up to ten times a day. This means that for the duration of the last sprint the entire had the latest code, bugs were identified quicker, and the program was being refined much faster than in the first sprint.

A large part of the sprint review in the previous sprint was discussion regarding how some of the agile tools were not being used to the full potential. Key points included not committing after a feature implementation, not testing using Gradle before pushing to GitHub, and not checking the results of the Jenkins Build on Discord. This sprint all members of the team were quite successful in resolving these issues. Firstly, the downtime of the build on Jenkins was significantly better (see Figure 2). All developers started to test code natively before pushing, and whenever a build did fail (and members were notified on discord) all members were quick to react. However, members did have issues with Git and GitHub. There were many times when the members were working on the same files, and commits reversed each other members commits. These meant that there was some reduction in efficiency while debugging pushes to GitHub. This was noted in the Sprint Review.

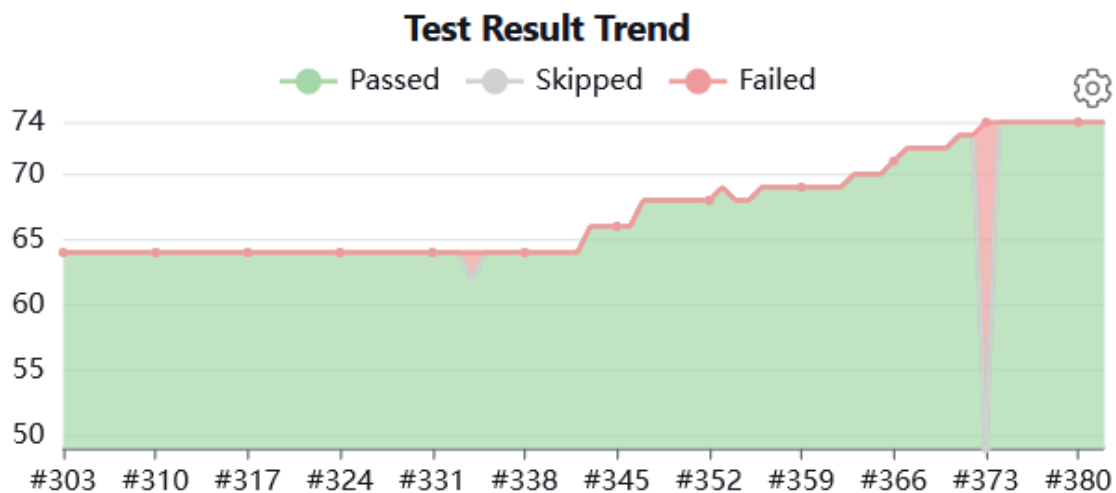


Figure 2: The test result trend chart for Sprint 3.

3.2 Scrum

Throughout the three sprints, our team grew better at our implementation of key Scrum features such as developing user stories, short and regular stand-up meetings, regular deployment to Jenkins through GitHub, and overall team communication.

A key misunderstanding of the Scrum method originally was that it involved a lot of real-time communication. Stand-up meetings lasted up to an hour each and usually involved the official “stand-up” plus a detailed, technical, almost code-review session afterwards where a team member would file what they’re having issues with. These long meetings actually detracted away from time which team members could have used to spend programming — a very common indictment on the Scrum method. Furthermore, communication which was actually necessary – such as letting another developer know when you were editing/changing their code – was not occurring frequently.

However, over the course of three sprints, our team landed a comfortable balance between time spent in active communication and time spent offline. This is part of why the team was completing a net higher amount of user story points in Sprint 3 as compared to Sprint 1.

Another issue the team faced was not being proactive in updating the status of their tasks on the Jira Scrum board. This is a similar issue to one discussed in the previous sprint’s Sprint Review where the team discussed the need to commit and push to Git more. That issue is now largely solved, however the number of accesses to the Jira board over the Sprint remains low. This is the reason behind the unfortunate Sprint Burndown Chart seen in Figure 3. Many tasks were actually completed throughout the week but not updated on Jira, reflecting poorly on the Burndown Chart.

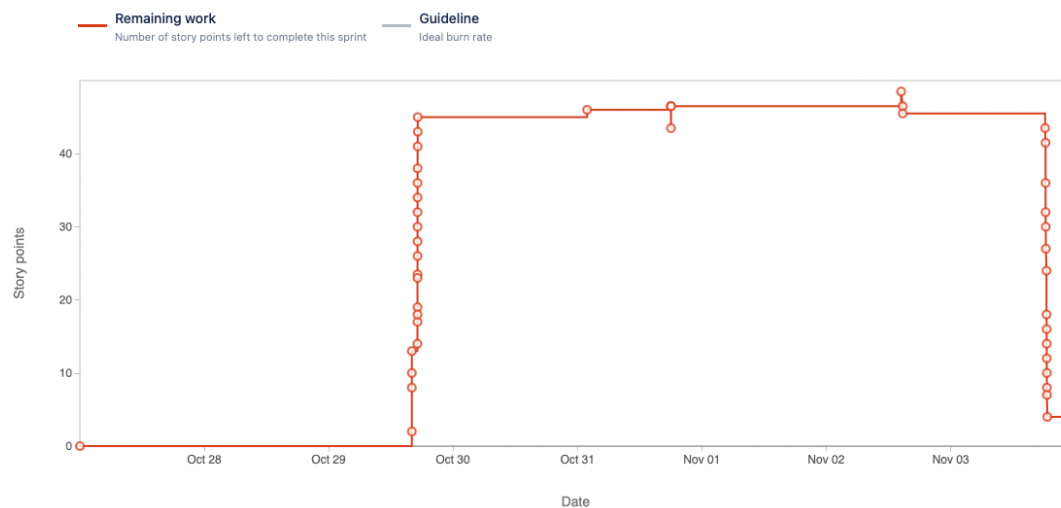


Figure 3: The burndown chart for Sprint 3

3.2.1 Sprint 3 Review - 03/11/2022

A sprint review was conducted at the end of the sprint, and overall the team felt good about the progress they made over the sprint. However, there are still a lot of areas the team has to improve on with the most prominent being clear and consistent communication.

The team for the final sprint attempted to implement all the user stories that were left however did not manage to do so. The user stories that haven't been implemented can be seen in Figure 4.

Although these user stories have been assigned to team members due to inefficient allocations, communication and poor time management the team did not manage to implement these user stories. Not all user stories were assigned catering to the workload of each member, which meant that towards the end of the sprint some team members had completed their respective user stories, while others were still working on their stories. The team believe this would've been avoidable if the team had communicated better.

Sprint 3 review

The meeting started at 5.20 PM.

Written By - Sulav

Attendees: Ankit, Antriksh, Udit, Sulav, Nemo

Feedback from Scrum Master:

- Udit said he had nothing notable in the feedback.
- Nemo did an amazing job on the Default page
- Everyone did the majority of their user stories.
- Once again the issue of prioritising the tasks appropriately, we could have re-distributed the tasks.
- People beside the product owner were changing the sprint backlog where they shouldn't have.
- Not all the user stories' acceptance criteria were done properly.
- Should've asked for help when needed, but overall good effort.
- As a team, we could've considered testing more.

Team review and general feedback from each member:

- In terms of tools, we did well, but we didn't communicate changes and check Jenkins properly.

- There were times when the build failed 5 times in a row and people didn't notice.
- Git and Gradle were used really well in general.
- Udit felt like he could've improved his testing and his consistency with his testing.
- Udit was pushing more often, and he felt we could've been more consistent with Java docs.
- Ankit felt the sprint has the work distributed really well over the week, he felt everyone did a good job. He has done all his user stories.
- Ankit felt that communication was a minor issue, and we could've trusted each other more as we are all capable.
- Ankit noticed a lot more merge conflicts, especially toward the end of the sprint, which is expected. These were handled well.
- Antriksh felt he did most work toward the end, and he felt he could have been more consistent.
- Antriksh felt agile may not always be the best, especially when communication is poor. It makes things more difficult, and more likely for people to forget issues and minor tasks. Also, small tasks may lead to poor code.
- Nemo felt sprint 3 went well for him.
- He did all his tasks, but he did encounter many issues. Which may have been due to lack to communication.
- Nemo felt he could have done more. He also felt the team should've implemented more design patterns.
- Sulav felt the work was hard and required a lot more consistent work.
- Sulav felt that communication could've improved a lot, and felt a lot of the problems that were brought up and discussed wouldn't have been an issue if the communication was better.
- Not all the user stories were completed, which is a reflection of the inconsistent workflow of the team and inefficient prioritisation/distribution of tasks.

Projects / SOFT2412_A2

SFA Sprint 3

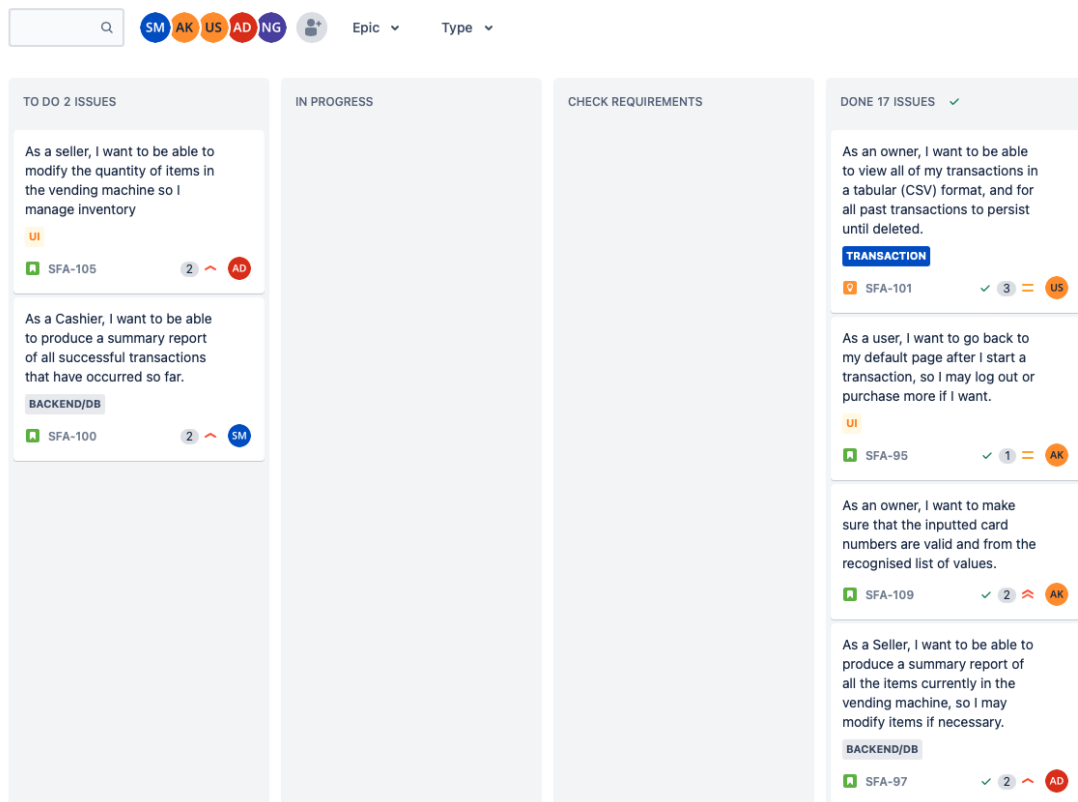


Figure 4: Kanban board for Sprint 3

3.2.2 Product Backlog

The product backlog had been setup from the previous sprint. Most user stories were already created before the sprint had started. As the sprint started the team added in acceptance criteria and story points. Then, throughout the sprint, priorities, statuses and progress were maintained using the sprint backlog.

3.2.3 Sprint Backlog

The sprint backlog was used to maintain and track all issues that the team was working through this sprint. This sprint of the 49 story points that the team had originally set out to complete, 45 were completed. Overall, the sprint backlog was maintained well. However, the team noted that sometimes that there was a delay in changing the status of the Jira issue after completion of the task.

3.3 Artefacts

During this current sprint a major focus, was storing and maintaining artefacts. The team originally implemented storing and maintaining the Jacoco Test Reports, and the Jar file, for all Builds. However, this was extended to store, the Gradle Test Report, and the Javadoc. Being able to view previous test reports allowed us to see and resolve recurring issues in the code base.



A note on the `.jar` executable file: due to inherent incompatibility issues with the JavaFX library, the `.jar` file created can only be run on the same operating system as the one it was compiled on. Hence, the file linked in the Canvas submission can only run on Mac.

Projects / SOFT2412_A2

Backlog

Search: AK SM AD US NG Epic Versions Type Insights

▼ SFA Sprint 3 27 Oct – 3 Nov (19 issues) 4 0 44.5 Complete sprint ...

SFA-101	As an owner, I want to be able to view all of my transactions in a tabular (CSV)...	TRANSACTION	3	=	DONE	US
SFA-95	As a user, I want to go back to my default page after I start a transaction, so I may log out or...	UI	1	=	DONE	AK
SFA-109	As an owner, I want to make sure that the inputted card numbers are valid and from the recogni...		2	⬆	DONE	AK
SFA-97	As a Seller, I want to be able to produce a summary report of all the items curren...	BACKEND/DB	2	⬆	DONE	AD
SFA-96	As an Owner, I want to able to produce a report summarising all the cancelled tr...	BACKEND/DB	2	=	DONE	AK
SFA-83	As a user, I want to be able to check-out so I can choose whether to pay with cash ...	SHOPPING	2	⬆	DONE	AK
SFA-84	As a user paying with cash, I want to be able to input a specific amount of currenc...	SHOPPING	6	⬆	DONE	SM
SFA-99	As a Cashier, I want to be able to produce a summary report of all available not...	TRANSACTION	3	⬆	DONE	SM
SFA-48	As a new customer (guest or registered) I want the previous customer's cart to be cleared a...	UI	2	⬇	DONE	NG
SFA-98	As a Seller, I want to be able to produce a summary report of all the items and t...	BACKEND/DB	2	⬆	DONE	AD
SFA-104	As a user, I want to complete transaction so I can see whether the transaction...	TRANSACTION	2	=	DONE	NG
SFA-103	As a User purchasing with card, I want my card numbers to be stored so that i...	TRANSACTION	4	⬆	DONE	AK
SFA-108	As an owner, I want my client's details to be encrypted using a salted hash to ...	BACKEND/DB	5.5	⬆	DONE	US
SFA-87	As a Cashier, I want to be able to view and update the amount of each denomination in the v...	UI	3	=	DONE	AD
SFA-107	As an Owner, I want to be able to add another user of any role so that I can dele...	BACKEND/DB	2	⬆	DONE	US
SFA-17	Create GUI Mock ups for the rest of the program	ADMIN	2	⬇	DONE	SM
SFA-93	As an owner, I want to remove a user from the vending machine application,...	AUTHENTICATION	1	⬆	DONE	US
SFA-105	As a seller, I want to be able to modify the quantity of items in the vending machine so I m...	UI	2	⬆	TO DO	AD
SFA-100	As a Cashier, I want to be able to produce a summary report of all successful t...	BACKEND/DB	2	⬆	TO DO	SM

+ Create issue

▼ Backlog (0 issues) 0 0 0 Create sprint

Your backlog is empty.

+ Create issue

Figure 5: Sprint Backlog for Sprint 3

4 Application Development

4.1 Features and Functionality

This sprint was extremely productive in terms of developing the functions and features outlined by the client at the start of the project. All developers worked together to fulfil the requirements, and as a result, the end product was successful from the developers' perspective. Some of the features added in this sprint include:

- Cash payments, which included:
 - A payment system that allowed the customer to choose from standard Australian currency denominations and pay for their items
 - An interface to display and dispense (from the database) the necessary amount of change from the inputs the user-specified
 - The ability to increase or decrease the quantity of a certain denomination in the machine
- Amendments to the card payment system, including:
 - Card detail storage and persistence upon exiting the application, and auto-filling when logging back into the application and making another transaction
 - Verification of card numbers using a list of verified details in the corresponding JSON file
 - Addition of all card payments and details to the database
 - Identification of the correct format for card numbers and CVVs (less than 16 digits and exactly 3 digits respectively), as requested by the client
- Refinement of the role designation system, using features such as:
 - Implementation of a salted hash algorithm to encrypt the passwords of all users
 - The ability for an owner to be able to remove a user from the application (and database)
 - The ability for an owner to be able to add a user to the application (and database)
- Amendments to the card payment system, including:
 - Card detail storage and persistence upon exiting and logging back into the application
 - Verification of card numbers using a list of verified details in the corresponding JSON file
 - Addition of all card payments and details to the database
 - Identification of the correct format for card numbers and CVVs, as requested by the client

4.2 Final Test Case Production

4.2.1 Test Reports

As this was the final sprint, the team needed to ensure that they had met the testing requirement of 75% coverage. Testing was primarily done towards the end of the sprint as all of the functions and methods were complete and could be fully tested without any amendments being needed.

Before the final presentation of the application to the client, a code coverage of 80% was achieved due to intensive testing being conducted between group members as the deadline neared. It was found that an effective approach to testing was to create test cases in groups of 2 developers to ensure that all edge cases were considered and the integration of various methods and elements from a different aspects of the application were tested holistically. The full coverage report can be found in Figure 6.

VendingMachine

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
Database		86%		90%	4	53	74	324	0	31	0	1
SceneManager		0%		n/a	20	20	38	38	20	20	1	1
Transaction		84%		78%	7	30	16	98	5	23	0	1
PasswordHash		84%		100%	2	10	9	40	2	8	0	1
Session		71%		100%	4	12	8	26	4	11	0	1
Main		0%		n/a	2	2	3	3	2	2	1	1
Password		100%		n/a	0	3	0	5	0	3	0	1
Total	446 of 2,274	80%	7 of 64	89%	39	130	148	534	33	98	2	7

Figure 6: The Jacoco Test Report for the Vending Machine application.

The Jenkins chart for testing can also be found in Figure 7.

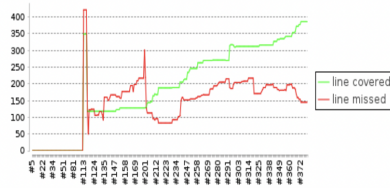
4.2.2 Exclusions from Test Reports

On the other hand, it was important to remove all of the scripts that did not need to be tested. This included all of the UI elements and the SceneManager script.

SceneManager did not need to be tested as it primarily relied on XML code and consisted mainly of getter and setter functions, both of which, by practice, are generally excluded from testing¹.

¹Do You Unit Test Getters and Setters? - No Fluff Just Stuff. 2022. Do You Unit Test Getters and Setters? - No Fluff Just Stuff. [ONLINE] Available at: https://nofluffjuststuff.com/blog/scott_leberknight/2007/05/do_you_unit_test_getters_and_setters_.

JaCoCo Coverage Report

[Download jacoco.exec binary coverage file](#)


Overall Coverage Summary

name	instruction	branch	complexity	line	method	class
all classes	81% M: 439 C: 1828	89% M: 7 C: 57	71% M: 37 C: 91	73% M: 145 C: 386	68% M: 31 C: 65	83% M: 1 C: 5

Coverage Breakdown by Package

name	instruction	branch	complexity	line	method	class
VendingMachine	M: 439 C: 1828 81%	M: 7 C: 57 89%	M: 37 C: 91 71%	M: 145 C: 386 73%	M: 31 C: 65 68%	M: 1 C: 5 83%

Figure 7: The Jenkins report for testing and code coverage for the Vending Machine application.

4.3 Application Design

4.3.1 Front-end user interface

The following is the Front End design of the final Vending Machine Application. In figure 8, the default page displays the categories on the right, the items in the centre and the cart on the right. Each item object has an image, to inform users of what the item looks like, they also have the item name, price, stock and an add-to-cart button, allowing users to create a cart of items to purchase.

After clicking the proceed to checkout button, the application proceeds to the checkout page as shown in figure 9. Here the user can select to pay with cash, card or cancel. There is also a 2min timer visible after which the transaction will automatically timeout.

If the user selects cash the cash page is displayed as seen in figure 10. The user can select what notes to enter and see the total amount, the amount due and change they will receive.

If the user selects Card the Card page is displayed as seen in figure 11. The user can enter their card number and CVV and it will be auto-filled if they saved it previously.

Upon a successful card or cash payment, the Transaction successful page is shown as seen in figure 12. The user can then stay logged in by clicking continue shopping or logout completely.

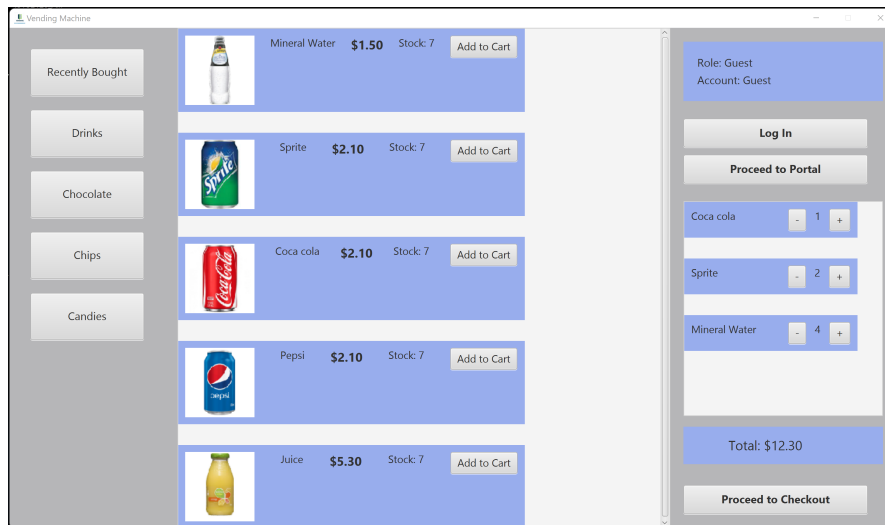


Figure 8: Default Page

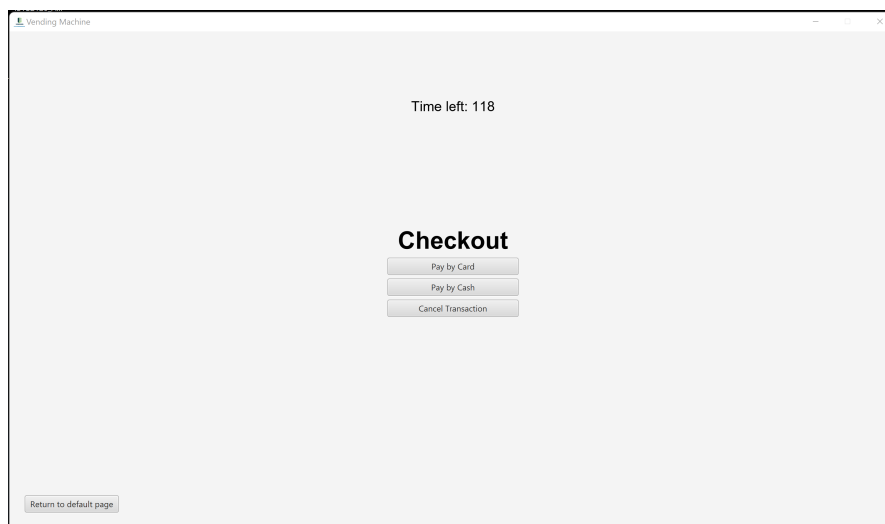
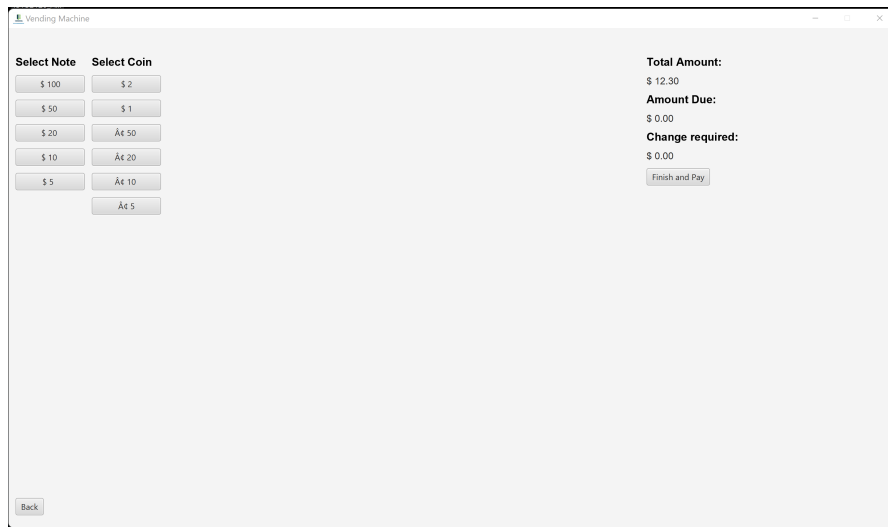


Figure 9: Checkout Page



The screenshot shows a web application window titled "Vending Machine". It features two columns of buttons for selecting payment: "Select Note" with buttons for \$100, \$50, \$20, \$10, and \$5; and "Select Coin" with buttons for \$2, \$1, Â€ 50, Â€ 20, Â€ 10, and Â€ 5. On the right side, the total amount is \$12.30, the amount due is \$0.00, and the change required is \$0.00. A "Finish and Pay" button is located below the change required section. A "Back" button is in the bottom left corner.

Select Note	Select Coin
\$100	\$2
\$50	\$1
\$20	Â€ 50
\$10	Â€ 20
\$5	Â€ 10
	Â€ 5

Total Amount:
\$ 12.30

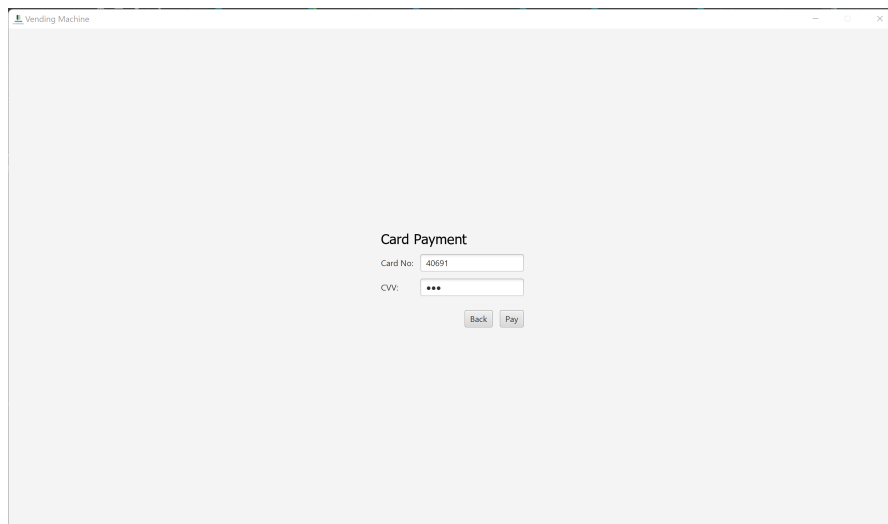
Amount Due:
\$ 0.00

Change required:
\$ 0.00

[Finish and Pay](#)

[Back](#)

Figure 10: Cash Page



The screenshot shows a web application window titled "Vending Machine". It displays a "Card Payment" section with input fields for "Card No:" (containing 40691) and "CVV:" (containing three dots). Below these fields are "Back" and "Pay" buttons.

Card Payment

Card No: 40691

CVV: ...

[Back](#) [Pay](#)

Figure 11: Card Page

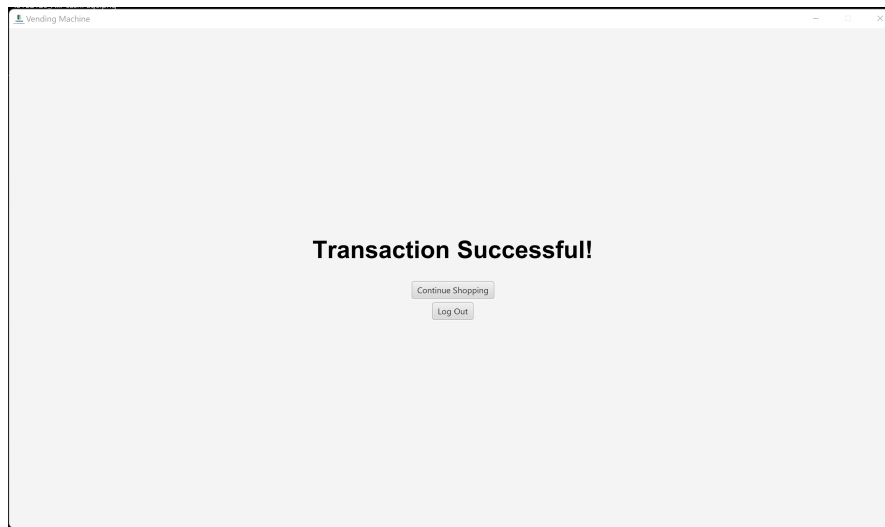


Figure 12: Transaction Complete Page

5 Overall Reflection

Over the course of the 3 sprints, the team has come to realise how difficult and stressful working in an agile environment can be. The amount of consistency, clear communication and teamwork required to was not something the team had expected at the beginning of this project. The team, although put in their best effort, they still believe it could have been done much better and more efficiently. The tools used by the team were not used optimally, for example not all jenkins builds were checked after each deployment meaning sometimes there would be build failure that would go unnoticed for a few days. Communication outside the stand-up meeting was very poor in general and a team member's need wouldn't be communicated to the team until near the end of the sprint. Also, the workflow was very inconsistent and not well paced as the team for most of the sprint ended up doing the majority of the work near the end of the sprint, and the deployment would be rushed.

A UML diagram

Please click [here](#) for a full size view of the UML class diagram. A copy has been placed below, split across Figures [13](#) and [14](#).

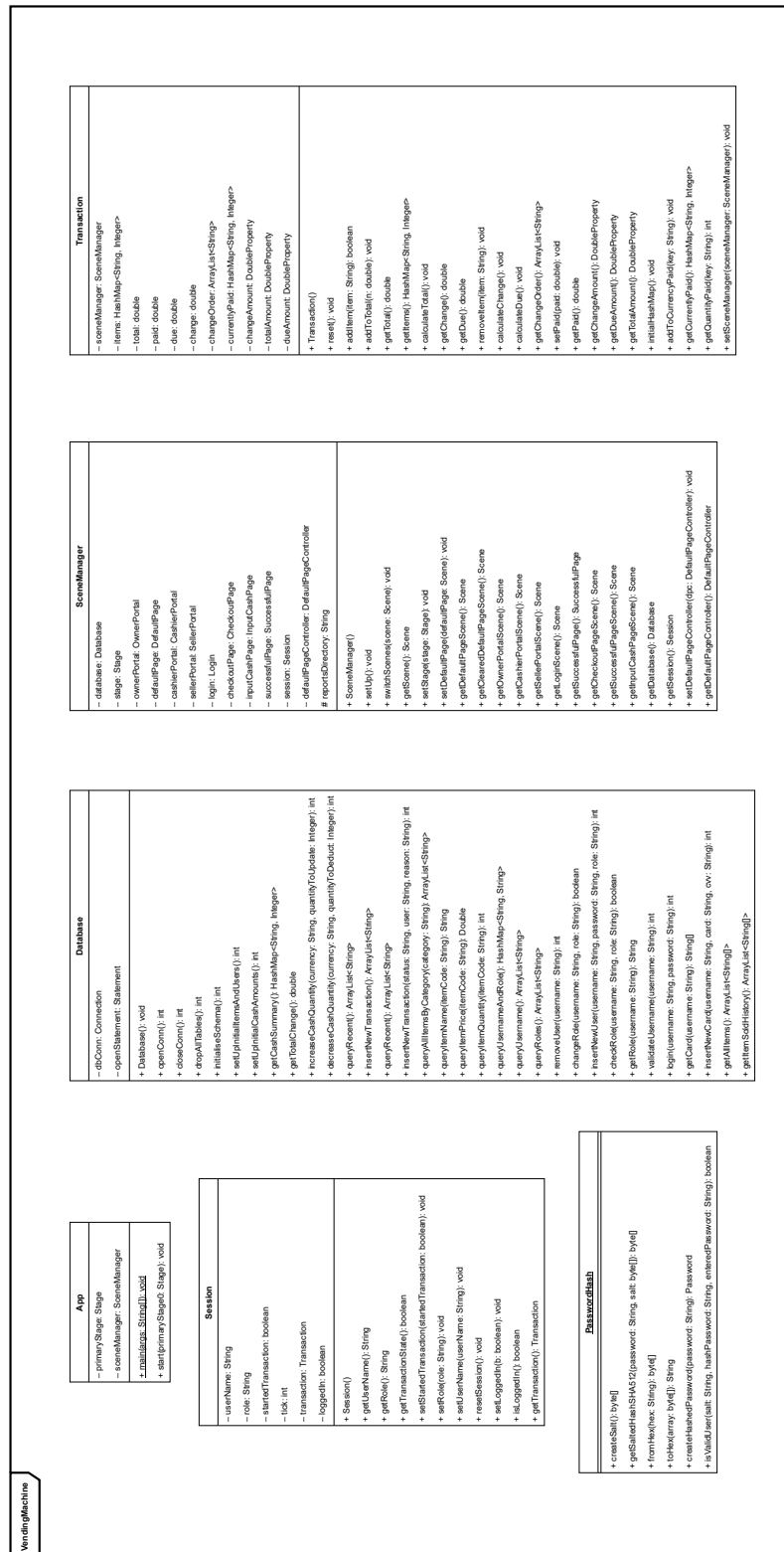


Figure 13: UML diagram part 1

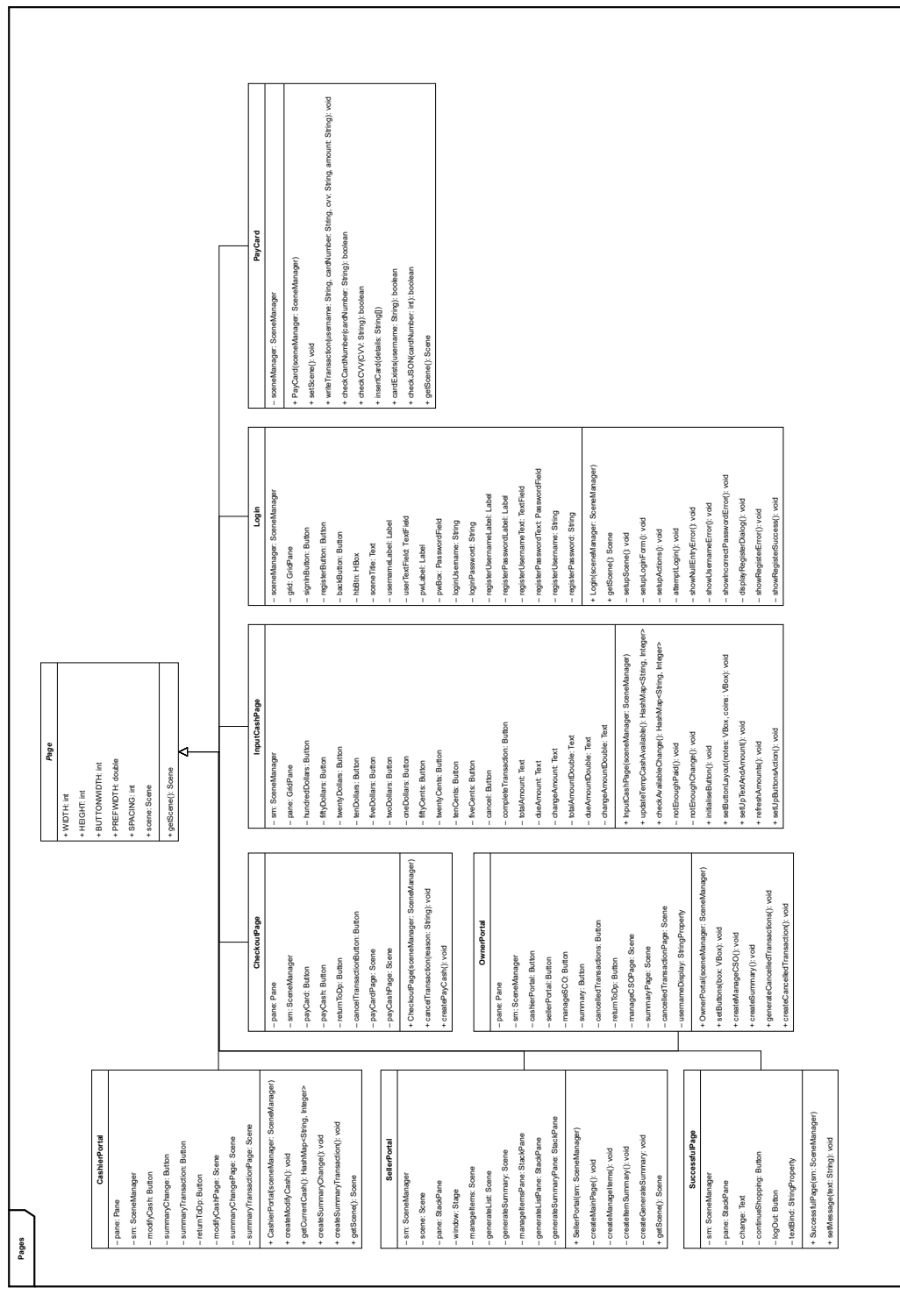


Figure 14: UML diagram part 2

B Meeting Minutes

Over the course of sprint 3, the team had a total of 4 standups. Each one lasted anywhere from 10 to 30 minutes and all coding team sessions were done separately. Realistically the team would've liked to have had a few more standups, but due to team busy schedules it was difficult to arrange meetings every day. Therefore it was cut down to 4 days.

These standups were all recorded and uploaded on the Confluence pages under "Meeting minutes". A copy of these is provided below.

Meeting minutes

30/10/2022

The meeting started at 11.00 PM.

Written By - Antriksh

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- Nemo created the "reset" function for Ankit, however it is not yet fully functional.
- Tomorrow he will work on his own user stories.
- Udit did not get to work on anything as he had an assignment due tonight.
- Udit gave moral support to Sulav for dynamic labels.
- Antriksh did not work on anything yet. He will work on this tomorrow.
- Sulav has finished writing his acceptance criteria up, and the mid-fi diagrams.
- The dynamic labels have been implemented by Sulav for cash payments.
- Udit asked the rest of the team to keep him accountable for completing the hashing of the passwords.
- Ankit did quite a lot of work today:
 - Got payment card system working
 - Pushes correct output to CSV file
 - Most of the acceptance criteria have been completed

31/10/2022

The meeting started at 11.00 PM.

Written By - Udit

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- Ankit has been working on his user stories, and has added all the Java docs too the function in the code base.

- Ankit plans on continuing to finish his user stories.
- Udit has started working on the salted hash password.
- Udit plan on finishing this over the next few day.
- Antriksh hasn't had the time to work on assignment, and will slowly start working on his user stories over the next few days.
- Sulav has slowly started working on the backend and database for the InputCashPage.
- Sulav will continue will working on his user stories for the time being.

01/11/2022

The meeting started at 11.00 PM.

Written By - Ankit

Attendees: Nemo, Ankit, Antriksh, Udit, Sulav

- Udit has almost completed his hashing.
- Udit plans on completing the hashing tomorrow and then do the testing for it.
- Sulav has linked all the buttons in the InputCash page.
- Sulav plans on developing the change algorithm tomorrow.
- Ankit has finished all of his user stories.
- Ankit found a user requirement that had been missed, card recognition from the provided JSON file, and plans on implementing this requirement by tomorrow.
- Antriksh has finished one of his user stories. He will do the rest tomorrow.

02/11/2022

The meeting started at 11.20 PM.

Written By - Sulav

Attendees: Ankit, Antriksh, Udit, Sulav

- Antriksh is has been working on his user stories, and now he is going to be working on the reports for seller.
- Ankit has finished his user stories, and he has helped out with some of Nemo's user stories.
- Ankit will continue finishing the payCard page and all the linking for the page.
- Udit has implemented how to make jar work with JavaFX, and has did everything else for the Jenkins build.
- Udit has built a successful jar file.
- Udit has finished with majority of his user stories, and he as implemented the hashing algorithm.
- Sulav has finished pay by cash with a few bugs which he plans to finish tomorrow.
- Sulav will continue working on his user stories.